



ITI · PYTHON FRAMEWORK WITH FASTAPI TRACK

Python Framework with FastAPI

Eng. Fatma Elraey*Lecture 1 — FastAPI Foundations — JSON-Only REST APIs*

Field	Detail
Instructor	Eng. Fatma Elraey
Course	Python Framework with FastAPI
Session	Lecture 1 of 5 — 1 Hour 48 Minutes
Focus	Application structure, route definitions, HTTP methods, parameters, JSON responses, and status codes.

Welcome!

FastAPI is treated throughout this course as a JSON-only API server. The goal is to build clean, testable REST endpoints that can be consumed by HTTP clients, JavaScript frontends, mobile apps, and other services. Server-side HTML rendering and templates are intentionally outside this course.

Today's Outline

1. FastAPI as a JSON-only API server
2. Application structure and route definitions
3. HTTP methods: GET, POST, PUT, DELETE
4. Path and query parameters
5. JSON responses and HTTP status codes

01 FastAPI as a JSON-Only API Server

FastAPI is a Python framework for building HTTP APIs. In this course the server returns JSON data, not server-rendered HTML. A browser, mobile app, JavaScript frontend, Postman, curl, or another backend can all consume the same endpoints.

JSON-only FastAPI request lifecycle



Figure 1.1 — A client sends HTTP; FastAPI routes the request and returns JSON.

- **REST resource thinking** — URLs represent resources such as `/documents` and `/documents/104`.
- **HTTP method** — the method communicates the operation: read, create, replace/update, or delete.

Before writing any code, we should be familiar with these 5 questions:

- URL?
- Method?
- Input?
- Output?
- Status Code?

Example:

- URL: `/documents/104`
- Method: `GET`
- Input: `document_id = 104`
- Output: JSON document
- Status: 200 if found , 404 if not found
- **JSON contract** — responses are machine-readable structures, not HTML templates.
- **Stateless request** — each request carries the information required for that operation.

```
# example — API vs server-rendered site
API GET /documents/104 → {"id": 104, "title": "FastAPI Notes"}
HTML SITE GET /documents/104 → <html><h1>FastAPI Notes</h1></html>
COURSE RULE We build the API version: JSON in, JSON out.
```

PRO TIP Think about the HTTP contract before writing endpoint code: URL, method, input, output, and status code.

✓ QUICK CHECK

- Q1.** What is the output format used throughout this course?
Answer: JSON responses.
- Q2.** Can the same API be consumed by a frontend and by curl?
Answer: Yes. Both are HTTP clients.

02

Application Structure & Your First Route

A minimal FastAPI application needs a FastAPI instance and at least one path operation. Uvicorn runs the ASGI application during development.

Minimal application

First:

```
pip install fastapi
```

```
# main.py
from fastapi import FastAPI

app = FastAPI(
    title="Documents API",
    version="1.0.0",
)

@app.get("/health")
def health_check():
    return {"status": "ok"}
```

Run the development server

```
python -m uvicorn main:app --reload
```

- **main** — the Python module main.py.
- **app** — the FastAPI object inside that module.
- **--reload** — restarts the development server after code changes.

To see output, go to terminal:

```
curl http://127.0.0.1:8000/health
```

or in the browser:

```
http://127.0.0.1:8000/health
```

FastAPI makes documentations automatically:

```
http://127.0.0.1:8000/docs
```

```
# example - route anatomy
@app.get("/health") route decorator: method + path
health_check() endpoint function
return {...} FastAPI serializes the Python dict to JSON
```

✓ QUICK CHECK

Q1. What does `@app.get("/health")` define?

Answer: A GET endpoint at /health.

Q2. Why can a Python dict be returned directly?

Answer: FastAPI serializes it into a JSON response.

03

HTTP Methods & Resource Operations

Method	Typical Meaning	Documents Example	Common Success
GET	Read	GET /documents	200 OK
POST	Create	POST /documents	201 Created
PUT	Replace full resource	PUT /documents/104	200 OK
PATCH	Partially update resource	PATCH /documents/104	200 OK
DELETE	Delete	DELETE /documents/104	200 OK or 204 No Content

In-memory resource example

```

from fastapi import FastAPI

app = FastAPI(
    title="Documents API",
    version="1.0.0",
)

@app.get("/health")
def health_check():
    return {"status": "ok"}

documents = {
    101: {
        "id": 101,
        "title": "Python Basics",
        "author": "Ahmed",
        "category": "Programming"
    },
    102: {
        "id": 102,
        "title": "Machine Learning Introduction",
        "author": "Sara",
        "category": "AI"
    },
    103: {
        "id": 103,
        "title": "Deep Learning Notes",
        "author": "Omar",
        "category": "AI"
    },
    104: {
        "id": 104,
        "title": "FastAPI Notes",
        "author": "Fatma",
        "category": "Backend"
    },
    105: {
        "id": 105,
        "title": "REST API Guide",
        "author": "Mohamed",
        "category": "Web Development"
    }
}

@app.get("/documents")
def list_documents():
    return {
        "items": list(documents.values())
    }

```

```
@app.delete("/documents/{document_id}")
def delete_document(document_id: int):

    removed = documents.pop(document_id, None)

    if removed is None:
        return {
            "error": "not_found"
        }

    return {
        "deleted": document_id
    }
```

04 Path Parameters & Query Parameters

Path parameters identify a specific resource. Query parameters modify how a collection is filtered, searched, sorted, or paginated.

Path + query parameters

```
from fastapi import Query

@app.get("/documents/{document_id}")
def get_document(document_id: int):
    return {"document_id": document_id}

@app.get("/documents")
def list_documents(
    q: str | None = Query(default=None),
    limit: int = Query(default=20, ge=1, le=100),
):
    return {"query": q, "limit": limit}
```

```
# example – how values arrive
PATH GET /documents/104 → document_id = 104
QUERY GET /documents?q=api&limit=10 → q="api", limit=10
```

If you want the items appearing using query parameters

```
@app.get("/documents")
def list_documents(q: str = None, limit: int = 10):
    items = list(documents.values())
    if q:
        items = [
            doc for doc in items
            if q.lower() in doc["title"].lower()
        ]
    items = items[:limit]
    return {
        "query": q,
        "limit": limit,
        "items": items
    }
```

PRO TIP Use the path for resource identity and query parameters for optional modifiers. Avoid turning every input into part of the URL path.

05 JSON Responses & Status Codes

HTTP status codes are part of the API contract. The JSON body explains the result; the status code communicates the outcome at the protocol level.

Code	Meaning	Typical FastAPI Use
200	OK	Successful GET/PUT/DELETE response
201	Created	Successful POST that created a resource
400	Bad Request	Semantically invalid request you reject explicitly
404	Not Found	Requested resource does not exist
422	Validation Error	Input failed FastAPI/Pydantic validation
500	Server Error	Unexpected server failure

Return the correct protocol-level result

```
from fastapi import HTTPException, status

@app.get("/documents/{document_id}")
def get_document(document_id: int):
    item = documents.get(document_id)
    if item is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Document not found",
        )
    return item
```

✓ QUICK CHECK

Q1. What status code should a successful create normally use?

Answer: 201 Created.

Q2. Why is 404 better than returning {"error": ...} with status 200?

Answer: Because the HTTP status communicates failure correctly to every client.

99

Check Your Understanding

Work through these on your own before checking the marked answers.

Practice Question 1

Which statement best describes this course's FastAPI server?

- A. It renders Jinja2 HTML pages
- B. It is a JSON-only REST API server
- C. It is a desktop GUI
- D. It only works with browsers

Practice Question 2

Which method is normally used to create a new resource?

- A. GET
- B. POST
- C. DELETE
- D. HEAD

Practice Question 3

In /documents/104, what is 104?

- A. A query parameter
- B. A header
- C. A path parameter
- D. A status code

Practice Question 4

What does 422 usually mean in FastAPI?

- A. Server crashed
- B. Input validation failed
- C. Resource deleted
- D. Request succeeded

Practice Question 5

Which response is most appropriate when a document does not exist?

- A. 200 with an error string
- B. 201
- C. 404
- D. 500

Answers

Practice Question 1: B ✓ — The server exposes HTTP endpoints and returns JSON.

Practice Question 2: B ✓ — POST commonly creates a new resource and often returns 201.

Practice Question 3: C ✓ — It identifies a specific resource in the URL path.

Practice Question 4: B ✓ — FastAPI/Pydantic uses 422 for validation failures.

Practice Question 5: C ✓ — 404 Not Found expresses that the resource is absent.